End of Term Report

Introduction

Gliomas, brain tumors arising from glial cells, remain among the most lethal cancers. Survival rates range from 5% to 95%, depending on tumor grade, subtype, and location. One of the most critical determinants of patient outcomes is the extent of surgical resection. However, neurosurgeons currently lack intraoperative tools capable of distinguishing cancerous tissue from healthy brain tissue in real time. Conventional imaging modalities such as MRI and CT, while indispensable for diagnosis and surgical planning, are impractical during surgery due to their cost, acquisition time, and vulnerability to motion and positioning errors. Fluorescence-guided imaging with 5-aminolevulinic acid (5-ALA) has improved intraoperative visualization but remains limited by timing constraints and poor sensitivity for low-grade gliomas.

Quantitative oblique back-illumination microscopy (qOBM) has recently emerged as a promising label-free imaging modality capable of generating high-resolution, 3D quantitative phase images in thick, scattering tissues in real-time. Because qOBM enables structural imaging without dyes and can be implemented in a compact fiber-based probe, it has significant potential for intraoperative tumor margin visualization. To support real-time visualization and data collection, our group has developed a MATLAB app that acquires, displays, and optionally saves qOBM frames from an imaging system during continuous acquisition.

A key restriction in the current system is the inability to save data efficiently while simultaneously displaying frames. Saving all frames at the end of an imaging session requires a large RAM buffer, which varies dramatically depending on the computer used. For example, a

system with 16 GB of RAM can save only ~1700 frames per run, whereas a 32 GB system can save ~7000 frames. This dependence limits the system's portability and imaging time, particularly for potential deployment in small devices like laptops, tablets, and mini-PCs for clinical settings.

The objective of this semester's work was to parallelize the MATLAB qOBM App to enable real-time acquisition and saving of frames, reduce memory usage, maintain GUI responsiveness, and support prolonged imaging. Several approaches were developed and evaluated, including sequential asynchronous saving (V1), parallel pool workers (V2), and worker-initialized acquisition (V3). This report details these implementations, the associated challenges, and their performance in enabling reliable high-speed qOBM imaging.

Methods

I. System Configuration

The compact qOBM system consists of a Basler industrial camera operating at 40 Hz in raw mode and 120 Hz in averaging mode, illuminated by four LEDs arranged at 90-degree intervals for oblique back-illumination. An electrically tunable lens (ETL) provides depth variation during acquisition. System control is executed in MATLAB App Designer, with dedicated buttons for continuous acquisition (StartContinuousButton) and data saving (SAVEButton). The Basler Pylon SDK and an Arduino microcontroller are used for frame acquisition and device control in order to synchronize LEDs. The application saves frames to disk based on the parallelization strategy used after processing each frame for display and optional averaging.

II. Original Workflow (Pre-Parallelization)

In the original system, all operations ran on the main MATLAB thread. Each frame was acquired, displayed, and, if saving was enabled, stored in a large RAM buffer. Only after acquisition ended were all buffered frames written to disk. This approach produced substantial GUI lag, required large amounts of memory, and limited total frames per session based on the system's RAM.

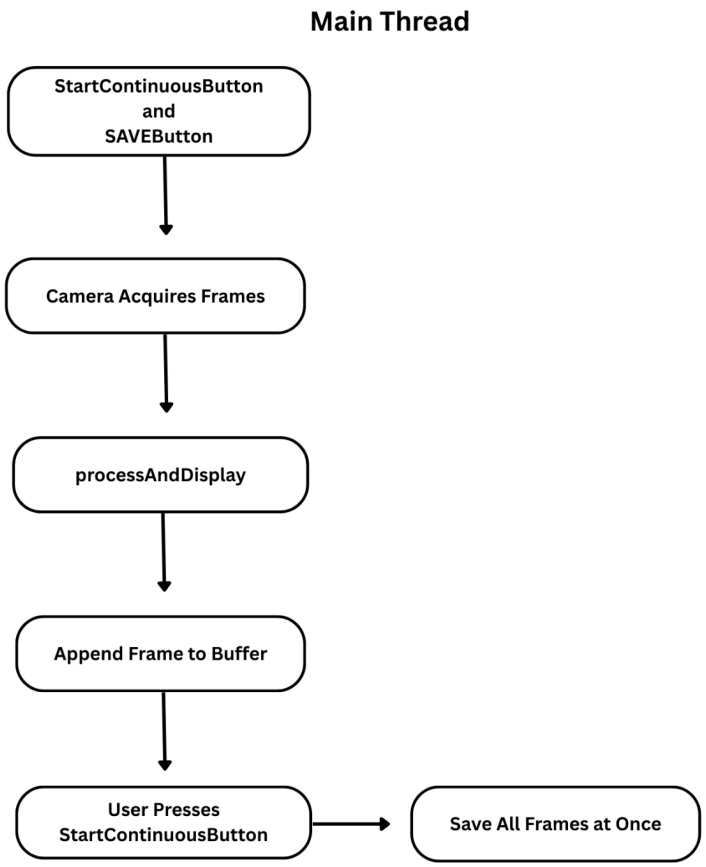

Figure 1. Original processing workflow. All acquisition, display, and saving occur on the main thread. Frames are stored in RAM until the end of the run, at which point they are written to disk.

III. Version 1: Sequential Asynchronous Saving

Version 1 introduced an improved control flow using three helper functions: routeFrames, listenPeek, and handleIncomingData. Crucially, metadata assignment occurs in routeFrames, where each frame is tagged with its cycle number, LED number, and ETL value. This metadata ensures correct temporal grouping for qOBM averaging. After the metadata is assigned, frames are routed simultaneously to listenPeek for display and handleIncomingData for averaging and immediate saving. This approach removes the need for a large RAM buffer and allows saving to occur continuously during acquisition, significantly extending the duration of imaging runs and maintaining responsiveness of the GUI.

However, at 120 Hz in averaged mode, Version 1 exhibits performance reduction. The rapid loop of metadata assignment, frame routing, display, averaging, and saving introduces timing offsets. As a result, frames from different acquisition cycles may be saved or averaged together. Because qOBM relies on precisely ordered LEDs, this desynchronization leads to metadata mismatch and degraded averaged images.

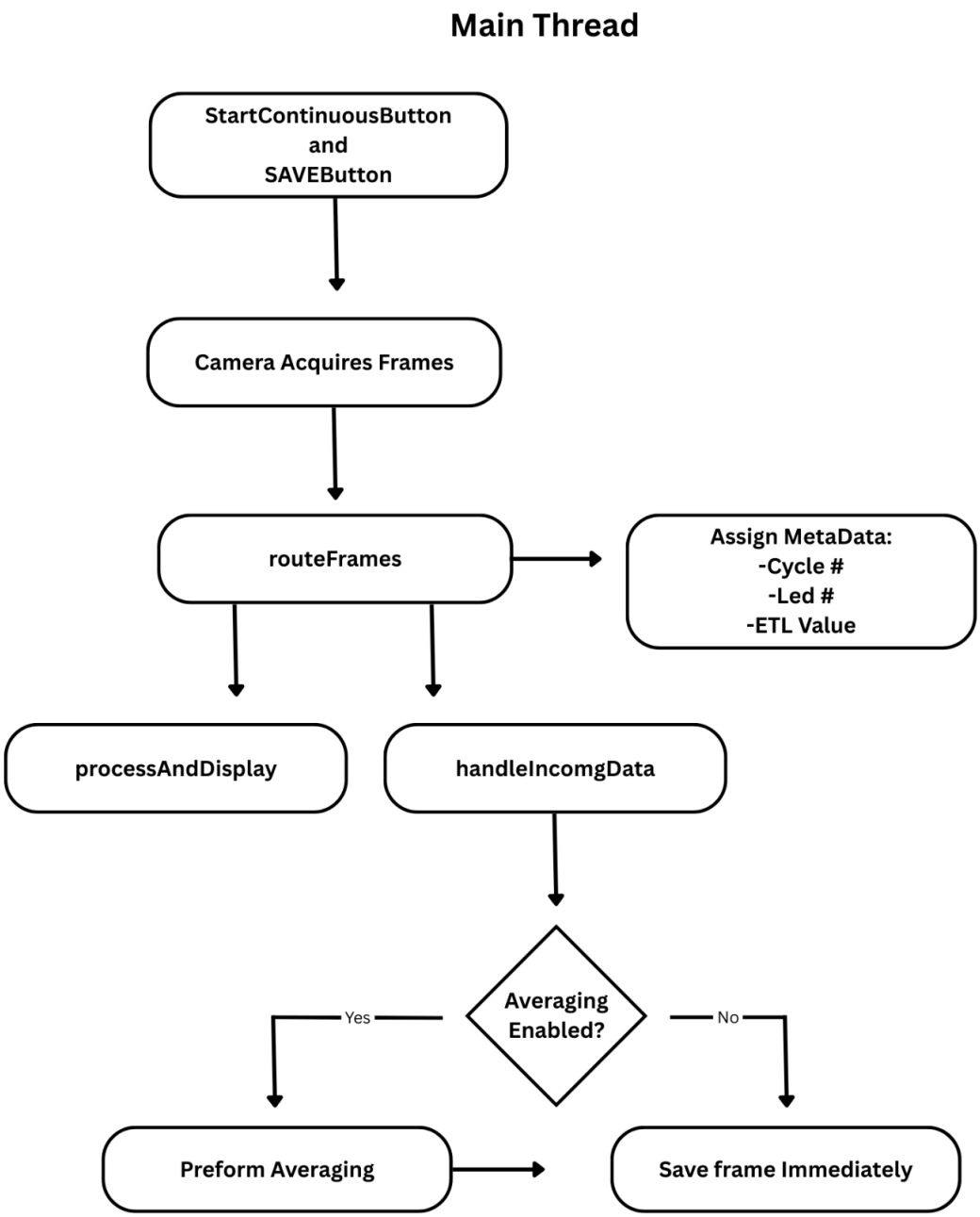

Figure 2. Workflow for Version 1. Metadata is assigned in routeFrames and frames are routed simultaneously for display and immediate saving.

IV. Version 2: Parallel Background Workers

Version 2 attempted to offload saving tasks to MATLAB's parpool by batching frames (typically every 200 frames) and sending them to a background worker. This approach theoretically allows acquisition and display to proceed uninterrupted on the main thread while saving occurs in parallel. In actual use, there was inconsistent communication between worker processes and the main MATLAB application. Due to an App Designer limitation, workers often failed silently, resulting in situations where frames were correctly displayed but not saved. Because debugging worker processes is limited in App Designer, reliably maintaining parallel saving was not achievable.

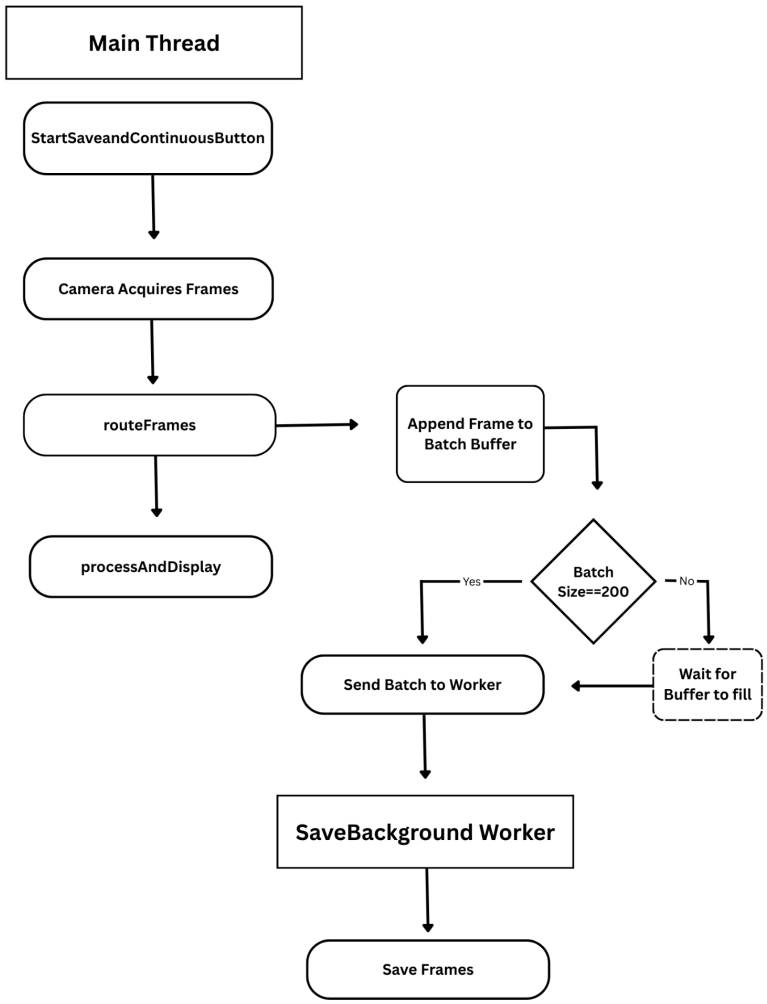

Figure 3. Workflow for Version 2. Batches of frames are periodically sent from the main thread to background workers for saving.

V. Version 3: Worker-Initialized Acquisition and Saving

To address worker-communication issues, Version 3 explored an architecture in which both the main thread and a background worker independently initialize camera and Arduino hardware. The main thread handles display, while the worker handles acquisition and saving. While conceptually appealing, this approach conflicts with hardware constraints. Industrial cameras and device drivers generally do not allow simultaneous access from two independent processes. As a result, this version produced inconsistent hardware behavior and remained too unstable for any qOBM acquisition.

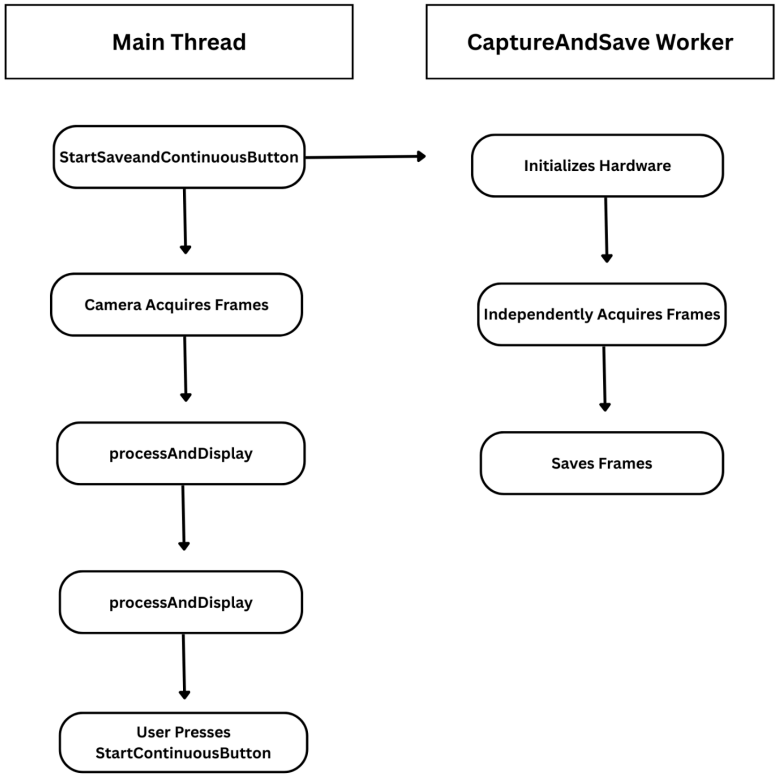

Figure 4. Workflow for Version 3. A background worker initializes and operates the camera independently of the main thread, which is responsible for display.

Results

I. Version 1 Performance

Among all implementations, Version 1 is the most stable and reliable. At 40 Hz raw acquisition, it supports sustained imaging with minimal memory usage and consistent frame saving. GUI responsiveness is maintained, and no worker-related issues occur since all operations remain on the main thread. However, at 120 Hz in averaged mode, Version 1 experiences metadata mismatch originating in routeFrames, where cycle number, LED number, and ETL value are assigned. Due to the speed at which frames are acquired and processed, the system saves frames faster than it can maintain synchronization between display, averaging, and metadata. Frames originating from different cycles can therefore be grouped together during averaging, or a saved frame may correspond to a previously displayed cycle. These mismatches produce visibly degraded qOBM images, characterized by blurring, reduced contrast, or distortions. Despite this limitation at high frame rates, Version 1 remains the most functional version and serves as the groundwork for more advanced asynchronous acquisition strategies.

II. Performance of Version 2 and Version 3

Versions 2 and 3 demonstrate promising directions but are limited by MATLAB App Designer's architecture. Background workers show unpredictable behavior when saving high-frequency data, and hardware initialization conflicts in Version 3 prevent reliable acquisition. Neither

version supports stable 120 Hz qOBM imaging, and both will require substantial restructuring or migration to lower-level programming environments to overcome these constraints.

Discussion

This work explored multiple strategies for implementing real-time, parallelized acquisition and saving for qOBM imaging. While Version 1 is the only fully functional and stable implementation, it remains limited by metadata synchronization at high acquisition rates. Versions 2 and 3 demonstrate the potential for more advanced parallelization but highlight fundamental constraints of MATLAB's App Designer and Parallel Computing Toolbox, especially in the context of high-throughput hardware control. The ultimate application of this work is the development of a compact, portable imaging platform suitable for intraoperative handheld use or deployment in resource-limited settings. A tablet-paired probe system could enable rapid, label-free identification of tumor margins during neurosurgery, improving surgical precision and expanding access to advanced imaging capabilities in low-income and under-equipped surgical centers.

References

Costa, P. C., Guang, Z., Ledwig, P., Zhang, Z., Neill, S., Olson, J. J., & Robles, F. E. (2021). Towards in-vivo label-free detection of brain tumor margins with epi-illumination tomographic quantitative phase imaging. Biomedical Optics Express, 12(3), 1621–1634. https://doi.org/10.1364/BOE.416731

Guang, Z., Ledwig, P., Costa, P. C., Filan, C., & Robles, F. E. (2022). Optimization of a flexible fiber-optic probe for epi-mode quantitative phase imaging. Optics Express, 30(11), 17713–17729. https://doi.org/10.1364/OE.454997

Ledwig, P., & Robles, F. E. (2019). Epi-mode tomographic quantitative phase imaging in thick scattering samples. Biomedical optics express, 10(7), 3605–3621. https://doi.org/10.1364/BOE.10.003605

Ledwig, P., & Robles, F. E. (2021). Quantitative 3D refractive index tomography of opaque samples in epi-mode. Optica, 8(1), 6–14. https://doi.org/10.1364/optica.410135

MathWorks. (n.d.). Perform Data Acquisition in Parallel with Postprocessing. MathWorks Documentation. Retrieved from https://www.mathworks.com/help/parallel-computing/perform-data-acquisition-in-parallel-with-postprocessing.html

MathWorks. (n.d.). Parallel Computing Toolbox Documentation. MathWorks. Retrieved from https://www.mathworks.com/help/parallel-computing/

Basler AG. (n.d.). Pylon SDK Documentation, Version 6.2. Retrieved from https://www.baslerweb.com/en/products/software/basler-pylon-camera-software-suite/

National Instruments. (n.d.). Asynchronous Data Acquisition and Multithreading Concepts. NI White Paper. Retrieved from https://www.ni.com/en/support/documentation/supplemental/06/archived--measurement-multithreading-in-ni-daqmx-and-traditional.html?srsltid=AfmBOopPkIHqnM92ZWQf3bhlsFWQUmh7wiDS4SVjjKwQclOZIBt2i